

Tema 1: fundamentos. Resumen y ejercicios

Diseño y Análisis de Algoritmos · Ciencia de la Computación · MatCom · 2026

Este documento cierra el Tema 1. No trae material nuevo. Junta en pocas páginas lo que las cinco conferencias contaron por separado, muestra las ideas que solo se ven mirándolas juntas, y termina con ejercicios que mezclan conferencias, en el estilo de las preguntas del examen final. Cuando algo se cita, se dice de qué conferencia y sección sale, para que puedas volver a la demostración completa.

1. El mapa del tema

La conferencia 2 organizó el trabajo en cinco preguntas: qué es el problema, qué algoritmo lo resuelve, si es correcto, cuánto cuesta y si se puede hacer mejor. Cada conferencia del tema dio la herramienta para una de ellas.

PREGUNTA	CONFERENCIA	HERRAMIENTA	EJEMPLO TRABAJADO	ERROR QUE ATRAPA
¿cómo se mide?	1	modelo RAM, notación asintótica, tamaño en bits	primalidad por división	llamar polinomial a algo exponencial en bits
¿qué algoritmo?	2	el ciclo completo	par más cercano	saltarse etapas del ciclo
¿es correcto?	3	invariante, potencial, primer fallo	Boyer-Moore, Gale-Shapley	el atajo del contador
¿cuánto cuesta?	4	análisis amortizado	cola, splay, union-find	subir a la raíz
¿se puede mejor?	5	información, adversario, reducción	ordenar, mezclar, segundo mayor	el torneo que ahorra una comparación

2. Lo que hay que llevarse de cada conferencia

Cada conferencia tiene su resumen al final. Aquí va una sola idea por conferencia: la que las demás usan.

Conferencia 1. Se mide en un modelo, y la entrada se mide en bits. Un algoritmo no cuesta segundos: cuesta operaciones en un modelo de cómputo. El modelo por defecto del curso es el word RAM, con palabras de $O(\log n)$ bits y operaciones de costo 1 sobre ellas. El tamaño de la entrada es la cantidad de bits que ocupa, y por eso probar divisores hasta $\sqrt{n}$ es exponencial aunque parezca un solo ciclo.

Conferencia 2. El ciclo de cinco preguntas, y la primera cota mínima. El par más cercano en el plano recorrió el ciclo entero: fuerza bruta en $\Theta(n^2)$, divide y vencerás en $\Theta(n \log n)$ con el lema de la franja, cota $\Omega(n \log n)$ por reducción desde distinción de elementos, y un algoritmo aleatorio de costo esperado lineal que la rompe. La conclusión fue que una cota mínima es una afirmación sobre un modelo.

Conferencia 3. Correcto quiere decir correctitud parcial más terminación. La parcial se demuestra con un invariante, que tiene que ser inductivo y no solo verdadero (Boyer-Moore). La terminación, con un potencial que baja (Gale-Shapley). Las propiedades globales del resultado, con el primer fallo: si algo sale mal, hay un primer momento en que salió mal, y ese momento es imposible (optimalidad de Gale-Shapley).

Conferencia 4. El costo amortizado es una garantía de peor caso sobre secuencias. No es un promedio y no tiene probabilidad. Se demuestra con un potencial que mide una deuda: los elementos sin mudar en la cola, los caminos largos en el splay tree, el crédito por grupos de rango en union-find. La suma funciona solo si el potencial termina al menos donde empezó.

Conferencia 5. La información cuenta salidas y el adversario cuenta trabajo. La información da $\lceil \log_2 n! \rceil$ para ordenar, pero solo $\log_2 n$ para el máximo. El adversario da las cotas exactas del máximo ($n - 1$), de mezclar ($2n - 1$) y del segundo mayor ($n + \lceil \log_2 n \rceil - 2$). Y una cota mínima demostrada sirve también para detectar algoritmos que están mal.

3. Cuatro ideas que atraviesan el tema

Toda cota viene con un modelo

La misma lección aparece cinco veces, cada vez con un modelo distinto:

- **Máquina de Turing contra RAM (C1):** la máquina de Turing define qué es computable, pero cambia los exponentes, porque acceder a la posición i de la cinta cuesta i pasos.
- **Multiplicación de costo 1 (C1):** una RAM con celdas sin límite que multiplica en un paso resuelve en tiempo polinomial problemas que se creen mucho más difíciles. Por eso las celdas son palabras.
- **Árboles algebraicos contra la rejilla (C2):** el par más cercano requiere $\Omega(n \log n)$ en árboles de decisión algebraicos, y el algoritmo aleatorio lo hace en tiempo esperado lineal usando la parte entera y una tabla hash, que ese modelo no tiene.
- **Sondeo de celdas (C4):** la cota de Fredman y Saks para union-find vale en un modelo que regala todo el cómputo y solo cobra los accesos a memoria.
- **Comparaciones contra conteo (C5):** ordenar requiere $\Omega(n \log n)$ comparaciones, y el ordenamiento por conteo ordena enteros pequeños en $O(n + k)$ usando los valores como índices.

Antes de citar una cota, pregunta cuál es su modelo y qué deja fuera.

Las pruebas no bastan, y la demostración dice dónde buscar

En cada conferencia desde la 2 plantamos un error en una variante que parecía correcta, y medimos cuánto lo ven las pruebas.

CONFERENCIA	ERROR PLANTADO	QUÉ VIERON LAS PRUEBAS	QUÉ LO ANUNCIABA
2, §7	1 vecino en lugar de 7 en la franja	19 fallos en 2000 instancias de $n = 12$, ninguno en 40 de $n = 400$	el lema de la franja
3, §5	devolver el candidato si el contador es positivo	0 % de fallos con mayoría plantada o alfabeto binario, 34 % con tres símbolos	la hipótesis "si hay mayoría"
4, §5-6	subir a la raíz en vez de splay	la cota del lema falla en 4 de 1500 accesos mezclados; en orden, $n/2$ por acceso	el caso zig-zig del lema de acceso
5, §8	el torneo que se salta un rival	falla con probabilidad $1/(n - 1)$: 0,12 % medido con $n = 1024$	la cota del adversario

Los cuatro casos tienen la misma forma. La variante pasa las pruebas que uno escribe primero, y falla en entradas con una estructura particular. La demostración nombra esa estructura: puntos amontonados con-

tra la recta de corte, arreglos sin mayoría, caminos recorridos en orden, el segundo mayor en la primera ronda. Probar verifica que implementaste lo que pensaste. La demostración verifica que pensaste bien, y además te dice qué probar.

Experimentar antes de demostrar

La otra cara de la idea anterior. Correr el algoritmo y mirar sirve para proponer el enunciado que después se demuestra:

- El experimento de 7 contra 1 vecino en la franja (C2, §7), leído al revés, sugiere que basta una constante pequeña, que es lo que el lema demuestra.
- El invariante vigilado de Boyer-Moore (C3, §4) comprueba en miles de prefijos que la partición en parejas se mantiene, antes de intentar demostrarlo.
- El lema de acceso ejecutado (C4, §5) calcula el potencial en cada acceso y muestra la cota cumpliéndose para splay y fallando para subir a la raíz.

El experimento propone y la demostración decide. Ninguno reemplaza al otro.

El caso típico no es la garantía

- En una nube de puntos uniforme, la franja casi nunca tiene ocho puntos, y el par que mejora a δ es casi siempre consecutivo (C2, §6).
- Gale-Shapley hace unas $n \ln n$ propuestas con listas al azar (243,7 para $n = 60$), no las n^2 de la cota (C3, §8).
- Una operación suelta de una estructura amortizada puede costar $\Theta(n)$: un desencolar que muda toda la pila, un acceso al fondo de un splay tree (C4, §10).

La garantía es la del peor caso, y el análisis tiene que cubrirlo aunque casi nunca ocurra.

4. Cómo se ve una respuesta completa

Un ejercicio del curso pide casi siempre lo mismo: el algoritmo, su correctitud, su costo y, cuando se puede, una cota mínima. El ejemplo que sigue es deliberadamente fácil, para que lo que se vea sea la forma de la respuesta.

Problema. Dado un arreglo A de $n \geq 1$ elementos de un orden total, devolver el máximo. Modelo: comparaciones.

Algoritmo. Recorrer el arreglo guardando el mayor visto hasta ahora.

```
def maximo(A):  
    m = A[0]  
    for x in A[1:]:  
        if x > m:  
            m = x  
    return m
```

Correctitud. Invariante: después de procesar $A[1..i]$, m es el máximo de $A[1..i]$. Inicialización: con $i = 1$, $m = A[1]$. Mantenimiento: si m es el máximo de $A[1..i]$, después de comparar con $A[i + 1]$ la variable tiene el mayor de los dos, que es el máximo de $A[1..i + 1]$. Conclusión: con $i = n$, m es el máximo de A . Termina porque el ciclo recorre una lista finita.

Costo. Exactamente $n - 1$ comparaciones, en todos los casos. Memoria $O(1)$.

Cota mínima. Todo algoritmo de comparaciones necesita $n - 1$. Si al terminar un elemento distinto del máximo declarado nunca perdió una comparación, se le puede subir el valor por encima de todos sin con-

tradecir ninguna respuesta, y el algoritmo habría devuelto un máximo equivocado. Así que los $n - 1$ elementos restantes pierden al menos una vez, y cada comparación tiene un solo perdedor (C5, §5).

Conclusión. El algoritmo es óptimo en el modelo de comparaciones.

Esa es la forma: enunciado con su modelo, algoritmo, invariante con sus tres pasos, terminación, costo exacto o asintótico, y cota mínima con su modelo. Una respuesta de examen no tiene que ser más larga que esto para un problema de este tamaño.

5. Ejercicios del tema

Cada ejercicio toca al menos dos conferencias, y lo dice entre paréntesis. La dificultad va de ★ a ★★. No llevan solución: como se dijo en la conferencia 1, las soluciones bien presentadas cuentan para la evaluación.

1. ★ Una computadora mil veces más rápida. ¿Cuánto crece el tamaño de entrada que se resuelve en una hora con un algoritmo $\Theta(n^2)$? ¿Y con uno $\Theta(n \log n)$? Estima los dos factores. (C1)
2. ★ Da el invariante y el potencial de la búsqueda binaria, y la cota de información que la hace óptima. (C3, C5)
3. ★ Una pila con una operación mínimo que cuesta $O(1)$ en el peor caso. Diseña la estructura, enuncia el invariante que la hace correcta y da el costo de cada operación. (C3, C4)
4. ★★ El contador binario en una máquina de Turing, del ejercicio 7 de la conferencia
 1. Demuestra con un potencial que m incrementos desde 0 cuestan $O(m)$ pasos en total, aunque uno solo pueda costar $\Theta(\log m)$. Es la pregunta que la conferencia 1 dejó abierta. (C1, C4)
5. ★★ Detección de ciclos en una lista enlazada con dos punteros que avanzan a velocidades 1 y 2 (algoritmo de Floyd). Demuestra que se encuentran si y solo si hay un ciclo, con un invariante, y que termina, con un potencial. Da el costo. (C3)
6. ★★ Una matriz $n \times n$ tiene filas y columnas ordenadas de forma creciente. Diseña un algoritmo que decida si contiene un valor x en $O(n)$ comparaciones, y demuestra con un adversario que no se puede en $o(n)$. (C2, C5)
7. ★★ Mediana de dos arreglos ordenados de largo n . Diseña un algoritmo $O(\log n)$, demuestra su correctitud y da la cota de información. (C2, C3, C5)
8. ★★ El elemento que aparece más de $n/3$ veces. Generaliza Boyer-Moore con dos candidatos, enuncia el invariante reforzado y demuéstralo. ¿Cuál sería el atajo equivocado análogo al de la conferencia 3, y qué generador de pruebas no lo detecta? (C3)
9. ★★ Máximo y mínimo a la vez con $\lceil 3n/2 \rceil - 2$ comparaciones, y la cota mínima por adversario. (C5)
10. ★★ Par más cercano en una dimensión. Demuestra $\Omega(n \log n)$ en árboles de decisión algebraicos por reducción desde distinción de elementos. Después di qué algoritmo de la conferencia 2 rompe la cota, en qué modelo, y adáptalo a una dimensión. (C2, C5)
11. ★★ Un arreglo dinámico que duplica al llenarse y se reduce a la mitad al quedar a un cuarto. Encuentra el potencial y demuestra $O(1)$ amortizado. Explica con una secuencia concreta por qué reducir a la mitad al quedar **a la mitad** cuesta $\Theta(n)$ por operación. (C4)
12. ★★ Union-find solo con compresión de caminos, sin unión por rango. Mide el costo promedio por operación sobre entradas que construyas para que sea malo, y compáralo con la cota de la conferencia 4. ¿Qué parte de la demostración de la conferencia 4 deja de valer? (C4)
13. ★★ Escoge un problema que no hayamos visto, recorre el ciclo completo de la conferencia 2 sobre él y consigue que el algoritmo y la cota mínima coincidan. Es el formato de la pregunta larga del examen final. (C2 a C5)